

Compliance in DevOps Environments: Case Study Analysis and Lessons Learned

Nima Memarzadeh

Module 12.2 Assignment

07/23/2025

Chapter 23 of The DevOps Handbook features two case studies examining compliance in DevOps settings. The two case studies show that conventional compliance approaches based on manual review and strict separation of duties don't work for fast-paced cloud environments. The case studies emphasize a move toward automated controls, continuous monitoring, and compliance through design, driven by engineering teams. Lessons from contemporary DevOps practice support these findings.

Case Study 1: Providing Compliance in Regulated Environments

AWS's principal security solutions architect Bill Shinn has noted the challenges of using traditional audit techniques in elastic cloud environments. Auditors typically ask for large samples of servers or screenshots in order to prove compliance with controls. However, such controls are hard to apply in a DevOps environment that deploys infrastructure as code and automatically scales hundreds of machines (Kim et al., 2016).

Shinn suggests engineering-based solutions to the problem. Involve auditors early in control design, use sprints to iterate, and then use telemetry tools (Kibana, Splunk, etc.) to provide real-time visibility into whether controls are working. He sees compliance as an engineering problem rather than a documentation problem, where control efficacy should be “discoverable by automated tooling somewhere in the delivery pipeline.”

Shinn also points out that there is a disconnect between compliance regulations and controls that are visible and testable. For example, achieving compliance with regulations like HIPAA are not achieved by stating good intentions but require technical controls that can be shown and tested. Logging access to patient data or setting up alerts for anomalies are typical ways to validate HIPAA compliance with technical means using infrastructure tools like AWS CloudWatch, log pushing mechanisms to central tools, and tracking between policies and code.

Lessons Learned

- Audit techniques that ask for samples or screenshots of servers do not work in cloud environments that change rapidly.
- Telemetry, logging, and other forms of automation provide real-time evidence that controls are working.
- Compliance is an engineering problem that should be built using collaboration between engineers and auditors.
- Controls need to be mapped to specific regulatory requirements then made testable and visible in engineering tooling.

These lessons are similar to a number of current articles on continuous compliance. For instance, Hazarika (2025) discusses a similar approach to compliance, where compliance is built into CI/CD pipelines with Policy as Code and real-time telemetry monitoring so that systems are always in compliance without slowing down deployment. Mishra and Wagle (2021) suggest an approach of using compliance tools in a validation and warning mode before switching to hard enforcement. Teams have time to adjust, which is similar to Shinn's idea of using sprints to design controls and gradually increase audit visibility.

Case Study 2: Relying on Production Telemetry for ATM Systems

Main Points

In this case, the attacker was a developer who leveraged a backdoor in ATM code. This backdoor allowed the user to gain access to ATM machines during unscheduled maintenance windows and then allowed them to dispense cash to themselves. This occurred outside of typical segregation of duties or change review boards (Kim et al., 2016). Instead, this attack was discovered as a result of a production monitoring review that happened to have someone look at and notice

suspicious behavior from the ATMs.

Key in this case is that solutions purely focused on prevention like code review and segregation of duties are not enough to combat this kind of attack. The means, motive, and opportunity existed for the fraudster, and the attacker was still able to succeed even with multiple static controls. Production telemetry and detection of these kinds of events is important. The other thing of note is that solutions that can see behavior in real-time seem to have the greatest success of actually catching these events.

Lessons Learned

- Production telemetry is an important aspect of the detection of fraud and errors.
- Solutions that see behavior in real-time seem to be more successful at stopping these types of events.
- Systems of compliance should be focused more on detection and response than pure prevention.
- Behavioral anomalies within a production environment serve as early warning signs for systemic issues

The key piece of this that is important to note is in how closely this lesson is seen in how organizations configure their continuous compliance workflow in the AWS cloud. For example, Mishra and Wagle (2021) detail how using the observation mode feature within AWS allows for potential compliance violations to be detected and resolved before they become production issues. In a similar way, solutions like Falco or AWS Config (Hazarika, 2025) allow organizations to detect drift and violations post-deployment in a similar way that the ATM team was able to.

Conclusion

The cases illustrate that legacy approaches based on large amounts of documentation, static controls, and code review are increasingly ineffective. The authors advocate for automated, continuous, and telemetry-based compliance approaches that are deeply integrated into DevOps workflows. This is strongly supported by a large body of modern tools and workflow principles described by experts in this field, such as Hazarika (2025) and Mishra & Wagle (2021). Controls that are designed to be easily testable and observable at a telemetry level and match regulatory needs well are enabling fast and secure software delivery.

References

Hazarika, K. (2025, May 19). *Implementing continuous compliance in DevOps pipelines*.

NimbusStack. <https://nimbusstack.com/implementing-continuous-compliance-in-devops-pipelines/>

Mishra, S., & Wagle, D. S. (2021, May 28). *Continuous compliance workflow for infrastructure as code: Part 1*. AWS Blogs. <https://aws.amazon.com/blogs/devops/continuous-compliance-workflow-for-infrastructure-as-code-part-1/>

Kim, G., Humble, J., Debois, P., Willis, J., & Allspaw, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press.